

## Exercises

### 6.3-1

Using Figure 6.3 as a model, illustrate the operation of BUILD-MAX-HEAP on the array  $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$ .

### 6.3-2

Show that  $\lceil n/2^{h+1} \rceil \geq 1/2$  for  $0 \leq h \leq \lfloor \lg n \rfloor$ .

### 6.3-3

Why does the loop index  $i$  in line 2 of BUILD-MAX-HEAP decrease from  $\lfloor n/2 \rfloor$  to 1 rather than increase from 1 to  $\lfloor n/2 \rfloor$ ?

### 6.3-4

Show that there are at most  $\lceil n/2^{h+1} \rceil$  nodes of height  $h$  in any  $n$ -element heap.

---

## 6.4 The heapsort algorithm

The heapsort algorithm, given by the procedure HEAPSORT, starts by calling the BUILD-MAX-HEAP procedure to build a max-heap on the input array  $A[1 : n]$ . Since the maximum element of the array is stored at the root  $A[1]$ , HEAPSORT can place it into its correct final position by exchanging it with  $A[n]$ . If the procedure then discards node  $n$  from the heap—and it can do so by simply decrementing  $A.heap\text{-}size$ —the children of the root remain max-heaps, but the new root element might violate the max-heap property. To restore the max-heap property, the procedure just calls MAX-HEAPIFY( $A, 1$ ), which leaves a max-heap in  $A[1 : n - 1]$ . The HEAPSORT procedure then repeats this process for the max-heap of size  $n - 1$  down to a heap of size 2. (See Exercise 6.4-2 for a precise loop invariant.)

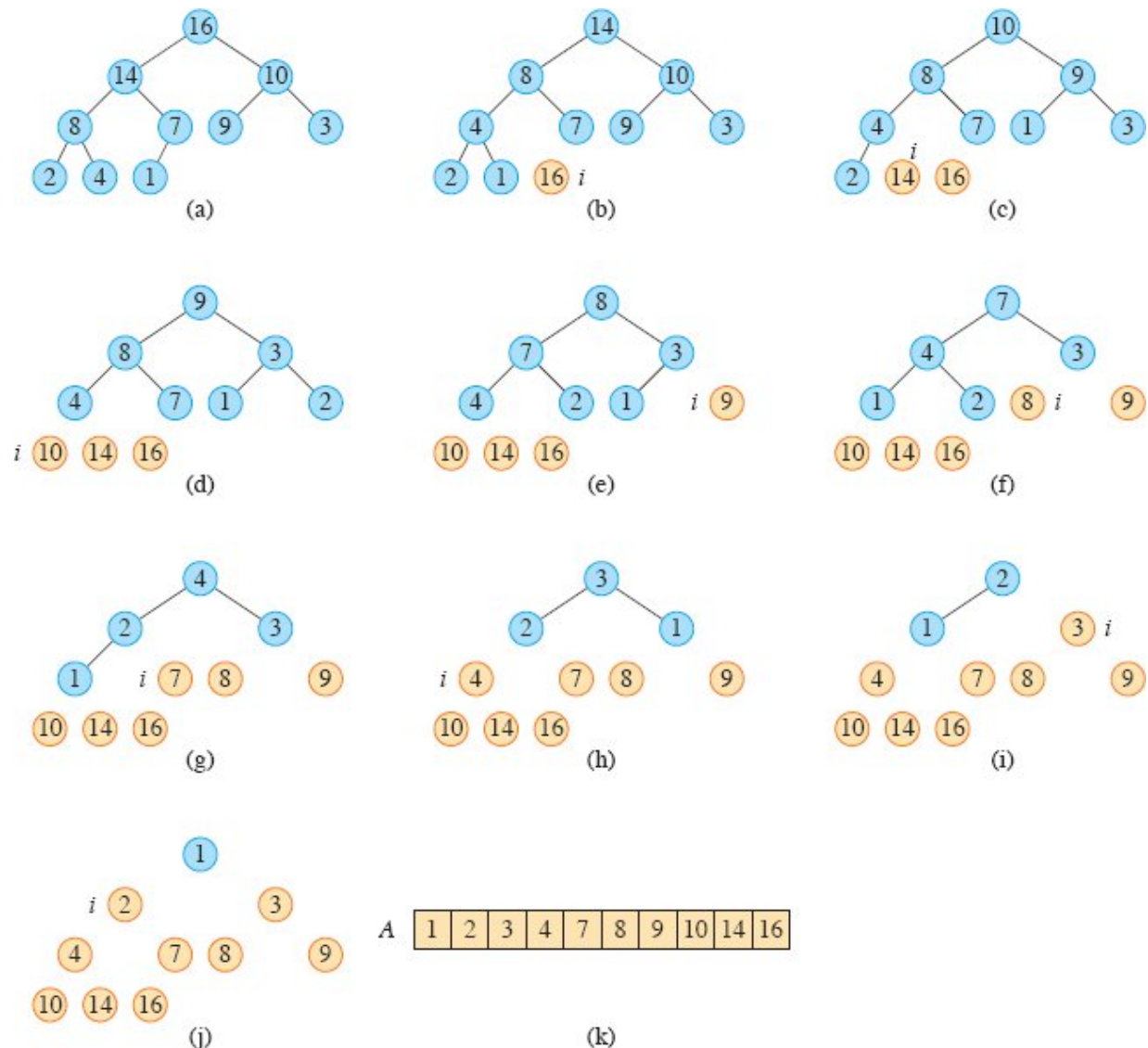
HEAPSORT( $A, n$ )

1 BUILD-MAX-HEAP( $A, n$ )

2 **for**  $i = n$  **downto** 2

```
3    exchange  $A[1]$  with  $A[i]$ 
4     $A.heap-size = A.heap-size - 1$ 
5    MAX-HEAPIFY( $A, 1$ )
```

Figure 6.4 shows an example of the operation of HEAPSORT after line 1 has built the initial max-heap. The figure shows the max-heap before the first iteration of the **for** loop of lines 2–5 and after each iteration.



**Figure 6.4** The operation of HEAPSORT. **(a)** The max-heap data structure just after BUILD-MAX-HEAP has built it in line 1. **(b)–(j)** The max-heap just after each call of MAX-HEAPIFY in line 5, showing the value of  $i$  at that time. Only blue nodes remain in the heap. Tan nodes contain the largest values in the array, in sorted order. **(k)** The resulting sorted array  $A$ .

The HEAPSORT procedure takes  $O(n \lg n)$  time, since the call to BUILD-MAX-HEAP takes  $O(n)$  time and each of the  $n - 1$  calls to MAX-HEAPIFY takes  $O(\lg n)$  time.

## Exercises

### 6.4-1

Using Figure 6.4 as a model, illustrate the operation of HEAPSORT on the array  $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$ .

### 6.4-2

Argue the correctness of HEAPSORT using the following loop invariant:

At the start of each iteration of the **for** loop of lines 2–5, the subarray  $A[1 : i]$  is a max-heap containing the  $i$  smallest elements of  $A[1 : n]$ , and the subarray  $A[i + 1 : n]$  contains the  $n - i$  largest elements of  $A[1 : n]$ , sorted.

### 6.4-3

What is the running time of HEAPSORT on an array  $A$  of length  $n$  that is already sorted in increasing order? How about if the array is already sorted in decreasing order?

### 6.4-4

Show that the worst-case running time of HEAPSORT is  $\Omega(n \lg n)$ .

### ★ 6.4-5

Show that when all the elements of  $A$  are distinct, the best-case running time of HEAPSORT is  $\Omega(n \lg n)$ .

---

## 6.5 Priority queues

In Chapter 8, we will see that any comparison-based sorting algorithm requires  $\Omega(n \lg n)$  comparisons and hence  $\Omega(n \lg n)$  time. Therefore, heapsort is asymptotically optimal among comparison-based sorting algorithms. Yet, a good implementation of quicksort, presented in Chapter 7, usually beats it in practice. Nevertheless, the heap data structure itself has many uses. In this section, we present one of the most popular applications of a heap: as an efficient priority queue. As with heaps, priority queues come in two forms: max-priority queues and min-priority queues. We'll focus here on how to implement max-priority